

# Geração Aumentada por Recuperação Aplicada a Bases de Código de Software: Um Estudo de Mapeamento Sistemático

Alex Ferreira de Almeida

Prof. Dr. Valdemar Vicente Graciano Neto

Programa de Pós-Graduação em Ciência da Computação (PPGCC)

Universidade Federal de Goiás (UFG) - Goiânia, GO, Brasil

almeida\_alex@discente.ufg.br, valdemarneto@ufg.br

## RESUMO

*Este artigo apresenta um Mapeamento Sistemático de Literatura (MSL) sobre abordagens de Geração Aumentada por Recuperação (RAG) baseadas em Modelos de Linguagem de Grande Escala (LLMs) aplicadas a bases de código de software. Seguindo as diretrizes de Petersen et al. (2015) e Kitchenham e Charters (2007), foram conduzidas buscas nas bases ACM Digital Library, IEEE Xplore e Scopus, cobrindo o período de janeiro de 2020 a 2026. A partir de 173 registros brutos, após remoção de 44 duplicatas, triagem de 129 artigos por título e resumo, e leitura integral de 58 estudos, foram selecionados 33 estudos primários. Os resultados revelam que detecção de vulnerabilidades é a tarefa-alvo mais frequente (9 estudos, ~27%), que modelos GPT da OpenAI dominam como componentes geradores, que Python e Java são as linguagens mais estudadas, e que generalização e arquitetura RAG constituem as principais categorias de lacunas identificadas. O mapeamento evidencia um campo em rápida expansão, com tendência crescente ao uso de RAG híbrido, agentes multi-step e chunking AST-aware, e aponta oportunidades de pesquisa em avaliação padronizada, raciocínio cross-file e suporte multilinguagem.*

**Palavras-chave:** Retrieval-Augmented Generation, Large Language Models, codebases, engenharia de software, mapeamento sistemático.

## ABSTRACT

*This paper presents a Systematic Mapping Study (SMS) on Retrieval-Augmented Generation (RAG) approaches based on Large Language Models (LLMs) applied to software codebases. Following the guidelines of Petersen et al. (2015) and Kitchenham and Charters (2007), searches were conducted in ACM Digital Library, IEEE Xplore, and Scopus, covering the period from January 2020 to 2026. From 173 raw records, after removing 44 duplicates, screening 129 articles by title and abstract, and fully reading 58 studies, 33 primary studies were selected. The results reveal that vulnerability detection is the most frequent target task (9 studies, ~27%), that OpenAI GPT models dominate as generator components, that Python and Java are the most studied languages, and that generalization and RAG architecture constitute the main categories of identified research gaps. The mapping evidences a rapidly expanding field, with a growing trend toward hybrid RAG, multi-step agents, and AST-aware chunking, and points to research opportunities in standardized evaluation, cross-file reasoning, and multilanguage support.*

**Keywords:** Retrieval-Augmented Generation, Large Language Models, codebases, software engineering, systematic mapping.

## 1. INTRODUÇÃO

A aplicação de Modelos de Linguagem de Grande Escala (LLMs) a tarefas de engenharia de software tem se intensificado de forma expressiva desde 2020, impulsionada pela disponibilidade de modelos pré-treinados em larga escala e pela crescente demanda por automação no ciclo de desenvolvimento de software. Entre as abordagens emergentes, a Geração Aumentada por Recuperação (RAG, do inglês *Retrieval-Augmented Generation*) destaca-se por combinar a capacidade generativa dos LLMs com a recuperação dinâmica de informações relevantes de bases de conhecimento externas, mitigando limitações inerentes aos modelos, como o conhecimento estático limitado pela data de corte do treinamento e a propensão a alucinações.

No contexto de bases de código de software, o RAG apresenta um potencial singular: ao indexar repositórios de código-fonte e recuperar trechos semanticamente relevantes em tempo de inferência, permite que o LLM produza respostas fundamentadas no código real do sistema, e não apenas em padrões genéricos aprendidos durante o pré-treinamento. Essa propriedade é especialmente relevante para tarefas como geração e complementação de código, detecção de vulnerabilidades, suporte ao desenvolvedor, revisão de código e transpilação entre linguagens.

Apesar do crescimento da literatura nessa interseção, não há, até o momento da condução deste estudo, um mapeamento sistemático que ofereça uma visão panorâmica e estruturada das abordagens, técnicas, modelos, linguagens e lacunas de pesquisa nesse campo específico. Trabalhos de revisão existentes tendem a abordar RAG de forma genérica ou a focar em subdomínios pontuais, sem o escopo abrangente necessário para orientar tanto pesquisadores quanto praticantes. O problema central que este estudo endereça é, portanto, a ausência de uma visão estruturada e abrangente sobre como RAG baseado em LLMs tem sido aplicado a *codebases* de software: quais arquiteturas são adotadas, quais modelos são empregados, quais linguagens e tarefas são cobertas, como a efetividade é avaliada e onde estão as lacunas que impedem o avanço do campo.

Este artigo preenche essa lacuna por meio de um Mapeamento Sistemático de Literatura (MSL), conduzido segundo as diretrizes de Petersen et al. (2015) e estruturado pelo framework PICOC de Kitchenham e Charters (2007). O objetivo central, formalizado pelo paradigma GQM (Basili, 1992), é analisar abordagens de RAG baseadas em LLMs aplicadas a *codebases* de software com o propósito de identificar e classificar suas técnicas, LLMs empregadas, linguagens de programação e tipos de sistemas estudados, métricas de avaliação e lacunas existentes, do ponto de vista de pesquisadores e praticantes de Engenharia de Software, no contexto de estudos primários publicados entre 2020 e 2026.

Para operacionalizar esse objetivo, foram definidas cinco questões de pesquisa:

RQ1: Quais técnicas e arquiteturas de RAG são propostas para aplicação em *codebases* de software?

RQ2: Quais LLMs são empregadas como componente gerador nas abordagens de RAG para *codebases*?

RQ3: Quais linguagens de programação e tipos de sistemas são mais estudados?

RQ4: Quais métricas são utilizadas para avaliar a efetividade das abordagens?

RQ5: Quais são as principais lacunas identificadas na literatura?

O restante do artigo está organizado da seguinte forma: a Seção 2 apresenta a fundamentação teórica sobre RAG e LLMs aplicados a código; a Seção 3 descreve o método de pesquisa; a Seção 4 apresenta os resultados por questão de pesquisa; a Seção 5 discute os achados e suas implicações; a Seção 6 trata das ameaças à validade; e a Seção 7 conclui o artigo.

## 2. FUNDAMENTAÇÃO TEÓRICA

### 2.1 Modelos de Linguagem de Grande Escala (LLMs)

LLMs são redes neurais baseadas na arquitetura Transformer (Vaswani et al., 2017), pré-treinadas em corpora massivos de texto com objetivos autossupervisionados, tipicamente predição do próximo token (modelos causais, como a família GPT) ou preenchimento de máscara (modelos bidirecionais, como BERT). A escala de parâmetros, que pode chegar a centenas de bilhões, combinada com o volume de dados de treinamento, confere a esses modelos capacidade emergente de generalização a tarefas não vistas, raciocínio em contexto (*in-context learning*) e geração de texto com alta fluência.

No domínio de código, LLMs especializados foram pré-treinados ou ajustados em grandes corpora de código-fonte, como GitHub, Stack Overflow e documentações técnicas. Exemplos incluem CodeBERT (Feng et al., 2020), UniXcoder (Guo et al., 2022), StarCoder (Li et al., 2023), CodeLlama (Rozière et al., 2023) e DeepSeek-Coder. Modelos de propósito geral com forte desempenho em código, como GPT-4 e Claude, também são amplamente empregados. A capacidade desses modelos de compreender e gerar código em múltiplas linguagens de programação ampliou significativamente o escopo de aplicações em Engenharia de Software.

### 2.2 Retrieval-Augmented Generation (RAG)

RAG é um paradigma arquitetural proposto por Lewis et al. (2020) que combina um componente de recuperação de informação (*retriever*) com um componente gerador (LLM). Em vez de depender exclusivamente do conhecimento paramétrico do modelo, o RAG recupera documentos ou trechos relevantes de uma base de conhecimento externa e os inclui no contexto da geração, permitindo respostas fundamentadas em informações atualizadas e específicas do domínio.

A arquitetura canônica de RAG envolve três etapas principais: (1) indexação, na qual os documentos da base de conhecimento são segmentados em *chunks*, convertidos em representações vetoriais (*embeddings*)

e armazenados em um *vector store*; (2) recuperação, na qual a consulta do usuário é convertida no mesmo espaço de *embedding* e os *chunks* mais similares são recuperados por busca aproximada de vizinhos mais próximos (ANN); e (3) geração, na qual o LLM recebe os *chunks* recuperados concatenados à consulta e produz a resposta final.

Variações desta arquitetura base incluem: RAG híbrido (combinando busca densa por similaridade semântica com busca esparsa léxica como BM25); GraphRAG (estruturando a base de conhecimento como grafo de entidades e relações); RAG *agentic* (incorporando agentes LLM que executam múltiplos passos de raciocínio e recuperação de forma iterativa); e RAG determinístico (substituindo a busca por similaridade por mecanismos estruturados como correspondência exata por rótulo ou consulta SQL).

### 2.3 Chunking e Abstract Syntax Tree (AST)

*Chunking* é o processo de segmentação de documentos em unidades menores (*chunks*) para indexação e recuperação em sistemas RAG. A granularidade e a estratégia de segmentação afetam diretamente a qualidade do contexto recuperado: *chunks* excessivamente pequenos perdem coesão semântica, enquanto *chunks* muito grandes extrapolam a janela de contexto do LLM e introduzem ruído na geração.

Para texto natural, a segmentação por janela deslizante (*sliding window*) com sobreposição parcial é a abordagem mais comum. Para código-fonte, essa estratégia é inadequada: cortar o texto em posições arbitrárias pode dividir funções, classes ou blocos lógicos ao meio, produzindo *chunks* sintaticamente inválidos e semanticamente incompletos.

A alternativa consolidada para código é o *chunking AST-aware*, que utiliza a Abstract Syntax Tree (AST) como estrutura de segmentação. A AST é uma representação hierárquica do código-fonte produzida pelo parser da linguagem, na qual cada nó representa uma construção sintática (função, classe, método, bloco condicional). Ao segmentar o código pelas fronteiras definidas pela AST, cada *chunk* corresponde a uma unidade sintática completa e autocontida, preservando a semântica operacional do trecho e tornando a recuperação mais precisa. Ferramentas como Tree-sitter fornecem parsers AST para múltiplas linguagens de programação e são amplamente utilizadas nos estudos do corpus deste mapeamento.

### 2.4 RAG Aplicado a Codebases

A aplicação de RAG a bases de código de software introduz desafios e oportunidades específicos em relação ao RAG sobre texto natural. O código-fonte possui estrutura sintática formal (definida por gramáticas de linguagens de programação), semântica operacional precisa e dependências complexas entre arquivos, módulos e funções. Essas características demandam estratégias de *chunking* adaptadas, como segmentação por unidades sintáticas via parsing AST (Abstract Syntax Tree), em contraste com a segmentação por janela deslizante comum em documentos de texto.

Adicionalmente, a recuperação de código relevante requer modelos de *embedding* sensíveis à semântica funcional do código. *Embeddings* especializados como CodeBERT e UniXcoder, treinados em pares (código, documentação), demonstram desempenho superior ao de *embeddings* de texto genérico em tarefas de *code search*. A combinação de *embeddings* densos com busca esparsa (BM25 ou TF-IDF) em configurações híbridas tem mostrado resultados consistentes em múltiplos estudos do corpus deste mapeamento.

O contexto *cross-file* representa um desafio particular: muitas tarefas de código requerem compreensão de dependências entre múltiplos arquivos de um repositório. Estratégias como sumarização hierárquica de repositórios (A40), recuperação baseada em grafos de dependência (A22) e agentes *multi-step* com ferramentas de análise estática (A57) emergem como abordagens promissoras para lidar com esse desafio.

### 2.5 Mapeamento Sistemático de Literatura

Um MSL é um método de revisão secundária de literatura que visa fornecer uma visão panorâmica de uma área de pesquisa, identificando a quantidade, tipo e distribuição de estudos disponíveis (Petersen et al., 2015). Diferencia-se da Revisão Sistemática de Literatura (RSL) por não agregar nem sintetizar quantitativamente os resultados dos estudos primários (meta-análise), focando na classificação e categorização do espaço de pesquisa. O processo de MSL segue cinco etapas: definição das questões de pesquisa; condução da busca; triagem por critérios de inclusão e exclusão; *keywording* para desenvolvimento do esquema de classificação; e extração e mapeamento dos dados.

## 3. MÉTODO DE PESQUISA

Este estudo segue as diretrizes de Petersen et al. (2015) para condução de MSL em Engenharia de Software. O protocolo de pesquisa foi definido a priori e registrado formalmente antes da execução da busca.

### 3.1 Objetivo e Questões de Pesquisa

O objetivo do estudo foi formalizado pelo paradigma GQM (Basili, 1992) conforme descrito na Seção 1. As cinco questões de pesquisa (RQ1-RQ5) foram derivadas das dimensões Outcome do framework PICOC, garantindo rastreabilidade direta entre os termos de busca e os objetivos da investigação.

### 3.2 Framework PICOC

A Tabela 1 apresenta a aplicação do framework PICOC ao escopo deste mapeamento. Os elementos Comparação e Contexto não compõem a string de busca, pois em MSLs o contexto é delimitado pelos critérios de seleção, e o mapeamento não pressupõe grupo controle.

*Tabela 1. Aplicação do framework PICOC.*

| Elemento    | Definição                                                                                                                                   |
|-------------|---------------------------------------------------------------------------------------------------------------------------------------------|
| População   | Codebases de software: código-fonte e repositórios de código.                                                                               |
| Intervenção | Abordagens de RAG baseadas em LLMs ou IA Generativa aplicadas a artefatos de código-fonte.                                                  |
| Comparação  | Não se aplica. O foco é catalogar o estado da arte, não comparar abordagens entre si.                                                       |
| Outcome     | Técnicas e arquiteturas de RAG; LLMs empregadas; linguagens de programação e tipos de sistemas; métricas de avaliação; lacunas de pesquisa. |
| Contexto    | Garantido pelos critérios de seleção. Delimitação à ES assegurada pelo CII; exigência de revisão por pares, pelo CE6.                       |

### 3.3 String de Busca

A string canônica foi construída a partir dos elementos População e Intervenção do PICOC, estruturada em três blocos conectados por AND:

*("source code" OR "source codes" OR "codebase" OR "code repository" OR "code search") AND ("Retrieval-Augmented Generation" OR "Retrieval Augmented Generation" OR "RAG") AND ("LLM" OR "Large Language Model" OR "GenAI" OR "Generative AI")*

A separação de RAG e LLM em dois blocos AND é uma escolha de precisão sobre sensibilidade: garante a recuperação apenas de estudos que tratam explicitamente da combinação RAG+LLM. O termo "Generative AI" foi incluído como hiperônimo para ampliar cobertura indireta de modelos citados por nome próprio sem os termos canônicos. A string foi adaptada à sintaxe específica de cada base (campos Title, Abstract e Keywords/Author Keywords).

### 3.4 Bases de Dados

Foram selecionadas três bases de dados seguindo as recomendações de Dyba, Kitchenham e Jorgensen (2005): ACM Digital Library, IEEE Xplore e Scopus. O Scopus foi incluído por sua ampla adoção posterior em revisões sistemáticas de Engenharia de Software (Kitchenham e Charters, 2007) e por indexar publicações das demais bases, ampliando a cobertura com sobreposição intencional.

### 3.5 Critérios de Inclusão e Exclusão

A Tabela 2 apresenta os critérios formais de seleção. Um único critério de inclusão (CI1) foi definido, com oito critérios de exclusão (CE1-CE8). Em caso de dúvida na triagem por título e resumo, a decisão padrão foi incluir, adiando a exclusão para a etapa de leitura integral, de forma a minimizar o viés de exclusão prematura (Petersen et al., 2015). CE3 e CE4 foram definidos a priori, mas não acionaram nenhuma exclusão: todos os 173 registros recuperados estavam em língua inglesa, e nenhum caso de mesmo trabalho publicado em múltiplos venues com revisão por pares foi identificado durante a leitura integral.

*Tabela 2. Critérios de inclusão e exclusão.*

| ID  | Critério de Inclusão                                                                                                                                                                                                                                                                                         |
|-----|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| CII | O artigo propõe, implementa ou avalia empiricamente uma abordagem de RAG baseada em LLM cujo foco principal seja a aplicação a artefatos de código-fonte (código de produção, documentação técnica gerada a partir de código, testes automatizados, scripts de infraestrutura ou queries de banco de dados). |
| ID  | Critério de Exclusão                                                                                                                                                                                                                                                                                         |
| CE1 | A abordagem descrita não constitui RAG baseado em LLM ou IA Generativa.                                                                                                                                                                                                                                      |
| CE2 | O foco principal não é a aplicação a artefatos de código-fonte.                                                                                                                                                                                                                                              |
| CE3 | Não redigido em língua inglesa.                                                                                                                                                                                                                                                                              |
| CE4 | Artigo duplicado: em caso de mesmo trabalho em múltiplos venues, manter apenas a versão publicada com revisão por pares; se ambas forem revisadas, manter a mais completa.                                                                                                                                   |
| CE5 | Artigo sem abstract disponível para leitura.                                                                                                                                                                                                                                                                 |
| CE6 | Publicação que não seja artigo de periódico científico ou artigo de conferência com revisão por pares formal: preprints, relatórios técnicos, whitepapers, posts, capítulos de livro, livros, teses, dissertações, etc.                                                                                      |
| CE7 | Publicado fora do período de janeiro de 2020 a 2026.                                                                                                                                                                                                                                                         |
| CE8 | Texto completo do artigo inacessível após busca nas bases institucionais (CAPES/UFG), Google Scholar e ResearchGate.                                                                                                                                                                                         |

### 3.6 Processo de Busca e Seleção

A busca foi executada em 02/06/2026, retornando 173 registros brutos distribuídos entre ACM Digital Library (n = 18), IEEE Xplore (n = 39) e Scopus (n = 116). Os metadados foram importados para a plataforma Rayyan (Ouzzani et al., 2016) para gerenciamento e rastreabilidade do processo de seleção.

Na etapa de remoção de duplicatas, o algoritmo do Rayyan identificou 43 duplicatas automaticamente; uma duplicata adicional ("N-CATS: A Neural Network Classification Automatic Taxonomy System") foi identificada manualmente durante a triagem, totalizando 44 registros removidos e 129 artigos únicos para avaliação.

Na triagem por título e resumo, os 129 artigos foram avaliados, resultando na exclusão de 67 estudos (CE1: 19, CE2: 44, CE6: 4) e no encaminhamento de 62 artigos para a etapa de elegibilidade.

Na etapa de elegibilidade, 4 artigos foram excluídos por inacessibilidade do texto completo após esgotamento das alternativas disponíveis: CAPES/UFG, Google Scholar e ResearchGate (CE8). Os 58 artigos restantes foram lidos integralmente, resultando na exclusão adicional de 25 estudos (CE1: 1, CE2: 24) e na seleção de 33 estudos primários para o corpus final.

A Tabela 3 resume o fluxo PRISMA do processo de seleção.

Tabela 3. Fluxo PRISMA do processo de seleção.

| Etapa                                | Detalhamento                       | n        |
|--------------------------------------|------------------------------------|----------|
| Identificação (registros brutos)     | ACM: 18   IEEE: 39   Scopus: 116   | 173      |
| Remoção de duplicatas                | 43 automáticas + 1 manual (N-CATS) | 44       |
| Triagem (título/resumo)              | CE1: 19   CE2: 44   CE6: 4         | 129 → 62 |
| Excluídos por inacessibilidade (CE8) | Antes da leitura integral          | 4        |
| Leitura integral                     | CE1: 1   CE2: 24                   | 58 → 33  |
| Corpus final                         | Estudos primários incluídos        | 33       |

A estratégia de *keywording* foi aplicada durante a extração de dados: os termos descritivos de cada estudo incluído foram analisados para derivar o esquema de classificação das questões de pesquisa, em conformidade com o método de Petersen et al. (2015).

#### 4. RESULTADOS

Esta seção apresenta os resultados do mapeamento para cada questão de pesquisa, com base nos 33 estudos primários incluídos.

##### 4.1 RQ1: Técnicas e Arquiteturas de RAG para *Codebases*

A análise dos 33 estudos revelou dois eixos classificatórios principais: a tarefa-alvo para a qual o sistema foi projetado, e o padrão arquitetural adotado. As categorias apresentadas nas Tabelas 4 e 5 foram derivadas indutivamente por meio da estratégia de *keywording* (Petersen et al., 2015), na qual os termos descritivos de cada estudo são analisados e agrupados progressivamente até a estabilização do esquema de classificação.

Em relação à tarefa-alvo, a Tabela 4 apresenta a distribuição dos estudos por categoria. Detecção de vulnerabilidades e segurança é a categoria mais frequente, com 9 estudos (~27%), refletindo a crescente preocupação com segurança de software no contexto de sistemas de IA. Q&A sobre *codebase* e suporte ao desenvolvedor (6 estudos) e *code completion* e geração de código (5 estudos) constituem as demais categorias de maior volume. Categorias como *code reuse*, *bug localization*, transpilação e auditoria de *smart contracts* emergiram como nichos com representação mais reduzida, mas com contribuições técnicas relevantes. Note-se que algumas categorias têm sobreposição intencional: A08 é contabilizado em detecção de vulnerabilidades e em *code review*; A10 e A49, em detecção e em auditoria de *smart contracts*; A27, em reuse e em transpilação.

Tabela 4. Distribuição dos estudos por categoria de tarefa-alvo (RQ1).

| Categoria de Tarefa-Alvo                                                                                 | n | Estudos                                     |
|----------------------------------------------------------------------------------------------------------|---|---------------------------------------------|
| Detecção de vulnerabilidades e segurança                                                                 | 9 | A06, A08, A10, A13, A14, A20, A28, A49, A55 |
| Q&A sobre <i>codebase</i> / suporte ao desenvolvedor                                                     | 6 | A02, A07, A32, A35, A47, A54                |
| <i>Code completion</i> e geração de código                                                               | 5 | A01, A05, A17, A22, A41                     |
| <i>Bug localization</i> e correção                                                                       | 4 | A16, A30, A40, A44                          |
| <i>Code reuse</i> , adaptação e <i>wiring</i>                                                            | 3 | A24, A27, A57                               |
| <i>Code review</i> e geração de comentários                                                              | 2 | A08, A23                                    |
| Transpilação de código                                                                                   | 2 | A27, A36                                    |
| Auditoria de <i>smart contracts</i>                                                                      | 2 | A10, A49                                    |
| Outros (FSM <i>inference</i> , <i>code recommendations</i> , RAG determinístico, categorização de erros) | 5 | A37, A43, A44, A50, A53                     |

Em relação aos padrões arquiteturais, a Tabela 5 classifica os estudos segundo a estratégia de RAG predominante. O RAG *agentic multi-step* (8 estudos) e o *chunking AST-aware* (7 estudos) são os padrões mais recorrentes, seguidos pelo RAG híbrido (6 estudos) e pelo RAG determinístico (5 estudos). Destaca-se a emergência do RAG determinístico: em vez de busca por similaridade vetorial, esses estudos empregam mecanismos estruturados como consulta SQL sobre esquemas relacionais orientados a objetos (A37), correspondência exata por rótulo de erro do compilador (A44), sumarização hierárquica de repositórios sem *embeddings* (A40), busca léxica BM25 pura (A50) ou recuperação via análise estática por compilador e AST parser (A57). Essa tendência indica que, para certos subdomínios de código, a estrutura formal da linguagem de programação oferece mecanismos de recuperação mais precisos do que a similaridade semântica vetorial.

Tabela 5. Padrões arquiteturais de RAG identificados (RQ1).

| Padrão Arquitetural                               | n | Estudos representativos                                                                               |
|---------------------------------------------------|---|-------------------------------------------------------------------------------------------------------|
| RAG <i>agentic multi-step</i>                     | 8 | A07, A08, A13, A22, A32, A49, A53, A57                                                                |
| <i>Chunking AST-aware</i>                         | 7 | A01, A07, A17, A35, A41, A47, A53                                                                     |
| RAG híbrido ( <i>dense + sparse</i> )             | 6 | A01, A16, A22, A41, A43, A55                                                                          |
| RAG determinístico (sem <i>embeddings</i> )       | 5 | A37 (SQL), A44 ( <i>exact match</i> ), A40 ( <i>summarization</i> ), A50 (BM25), A57 (AST+compilador) |
| RAG com <i>knowledge base</i> estruturada externa | 4 | A06, A44, A49, A55                                                                                    |
| GraphRAG                                          | 2 | A07, A22                                                                                              |

Achado RQ1: Detecção de vulnerabilidades é a tarefa-alvo dominante (~27%). RAG *agentic multi-step* e *chunking AST-aware* são os padrões arquiteturais mais recorrentes, com emergência do RAG determinístico como abordagem especializada para subdomínios estruturados de código.

## 4.2 RQ2: LLMs Empregadas como Componente Gerador

A Tabela 6 apresenta a distribuição das famílias de LLMs identificadas nos 33 estudos. A família GPT da OpenAI é amplamente dominante: modelos GPT-4, GPT-4o e GPT-3.5 Turbo aparecem em mais de 20 estudos, seja como componente gerador principal ou como baseline de comparação. Essa predominância reflete tanto o desempenho superior desses modelos nas tarefas avaliadas quanto a facilidade de acesso via API durante o período coberto pelo mapeamento.

Modelos *open-source* emergem como alternativas relevantes, especialmente as famílias DeepSeek, Qwen e LLaMA/CodeLlama. Modelos especializados em código, como CodeBERT e UniXcoder, são utilizados principalmente como encoders para geração de *embeddings* no componente de *retrieval*, não como geradores de texto.

Tabela 6. LLMs empregadas como componente gerador (RQ2).

| Família / Modelo                                                  | Tipo   | n      | Estudos                                                                                                                          |
|-------------------------------------------------------------------|--------|--------|----------------------------------------------------------------------------------------------------------------------------------|
| GPT-4 / GPT-4o / GPT-3.5 (OpenAI)                                 | Closed | >20    | A01, A02, A06, A07, A08, A10, A13, A14, A16, A20, A22, A23, A24, A27, A28, A35, A36, A40, A41, A47, A49, A53, A54 (entre outros) |
| LLaMA / CodeLlama (Meta)                                          | Open   | 6      | A06, A07, A14, A28, A43, A47                                                                                                     |
| DeepSeek / DeepSeek-Coder                                         | Open   | 5      | A01, A17, A41, A43, A57                                                                                                          |
| Qwen / Qwen2.5 / Qwen2.5-Coder                                    | Open   | 2      | A55, A57                                                                                                                         |
| Gemini (Google)                                                   | Closed | 3      | A16, A20, A49                                                                                                                    |
| Claude (Anthropic)                                                | Closed | 1      | A49                                                                                                                              |
| Modelos especializados em código (CodeBERT, UniXcoder, StarCoder) | Open   | Vários | A01, A16, A17, A27, A28, A41                                                                                                     |

A tendência observada é a concentração em modelos *frontier closed-source* como geradores principais, com modelos *open-source* como alternativas de menor custo ou maior privacidade. Apenas um estudo (A49) menciona Claude (Anthropic) como modelo de comparação, e três estudos (A16, A20, A49) avaliam modelos Gemini. Essa assimetria pode introduzir viés nas conclusões sobre efetividade relativa dos LLMs para tarefas de código.

Achado RQ2: Modelos GPT da OpenAI dominam como componente gerador (>20 estudos). Modelos *open-source* como DeepSeek-Coder e CodeLlama emergem como alternativas relevantes, mas a dependência de infraestrutura *closed-source* permanece predominante.

## 4.3 RQ3: Linguagens de Programação e Tipos de Sistemas

A Tabela 7 apresenta a distribuição das linguagens de programação nos estudos primários. Python é a linguagem mais estudada (10 estudos), seguida por Java (8 estudos) e C/C++ (7 estudos). A predominância de Python reflete sua centralidade em *pipelines* de dados, *machine learning* e desenvolvimento web. C/C++ concentra-se em estudos de detecção de vulnerabilidades e segurança de sistemas. Java aparece com frequência em contextos de refatoração, *code reuse*, *bug localization* e aplicações empresariais.

Tabela 7. Linguagens de programação nos estudos primários (RQ3).

| Linguagem / Tecnologia             | n estudos | Estudos                                          |
|------------------------------------|-----------|--------------------------------------------------|
| Python                             | 10        | A05, A07, A14, A22, A28, A32, A35, A41, A47, A50 |
| Java                               | 8         | A20, A24, A32, A37, A40, A41, A43, A57           |
| C / C++                            | 7         | A06, A17, A23, A27, A28, A53, A55                |
| JavaScript / TypeScript            | 4         | A08, A10, A47, A54                               |
| Solidity                           | 3         | A10, A49 (e A08 como linguagem secundária)       |
| Verilog HDL                        | 2         | A30, A44                                         |
| Multi-linguagem (3 ou mais)        | 4         | A08, A32, A41, A47                               |
| DSLs (Ansible-YAML, Lingua Franca) | 2         | A05, A36                                         |

Linguagens de hardware (Verilog HDL, 2 estudos) e de domínio específico (Ansible-YAML, Lingua Franca) indicam que a aplicação de RAG a *codebases* está se expandindo além das linguagens de propósito geral. Linguagens como Rust, Go, Kotlin e Swift têm presença marginal ou aparecem apenas como targets de transpilação, indicando lacuna de cobertura para linguagens de crescente adoção industrial.

Achado RQ3: Python (10 estudos), Java (8) e C/C++ (7) são as linguagens mais estudadas. Linguagens de crescente adoção industrial como Rust, Go e Kotlin têm presença marginal no corpus.

#### 4.4 RQ4: Métricas de Avaliação

A Tabela 8 apresenta a distribuição das categorias de métricas identificadas. Métricas de classificação (Precision, Recall, F1-Score) dominam com 18 estudos, reflexo direto da predominância de tarefas de detecção de vulnerabilidades e malware no corpus. Avaliação humana aparece em 9 estudos, frequentemente combinada com avaliação automática para validar aspectos de qualidade não capturáveis por métricas quantitativas.

Tabela 8. Categorias de métricas de avaliação (RQ4).

| Categoria de Métrica                                    | n estudos | Estudos                                                                                  |
|---------------------------------------------------------|-----------|------------------------------------------------------------------------------------------|
| Classificação: Precision, Recall, F1-Score, MCC         | 18        | A06, A08, A10, A13, A14, A16, A17, A20, A28, A43, A44, A47, A49, A50, A53, A54, A55, A57 |
| Avaliação humana: Likert, expert rating, user study     | 9         | A02, A17, A23, A30, A32, A35, A47, A54, A57                                              |
| Métricas de código: BLEU, CodeBLEU, pass@k, Exact Match | 8         | A01, A05, A22, A27, A30, A36, A37, A41                                                   |
| LLM-as-a-Judge                                          | 5         | A23, A24, A35, A50, A55                                                                  |
| Métricas de retrieval: MRR, P@k, Recall@k               | 5         | A01, A35, A41, A47, A49                                                                  |

Métricas de código como BLEU, CodeBLEU e pass@k aparecem em 8 estudos de geração de código, mas com limitações conhecidas: BLEU mede sobreposição léxica sem capturar correção funcional, e pass@k requer execução de casos de teste, nem sempre disponíveis. O uso crescente de *LLM-as-a-Judge* (5 estudos) como avaliador automático de qualidade é uma tendência recente que pode mitigar limitações de métricas puramente léxicas, mas introduz dependência de um segundo modelo potencialmente tendencioso. A heterogeneidade das métricas adotadas dificulta comparações diretas entre os estudos, mesmo dentro de uma mesma categoria de tarefa.

Achado RQ4: Métricas de classificação (Precision, Recall, F1) dominam com 18 estudos. A heterogeneidade de métricas e a ausência de *benchmarks* padronizados para RAG sobre código dificultam comparações entre abordagens.

#### 4.5 RQ5: Lacunas de Pesquisa

A Tabela 9 apresenta as lacunas identificadas nos 33 estudos, organizadas por categoria e frequência de ocorrência. Generalização e arquitetura RAG são as categorias mais críticas, com 38 e 35 ocorrências respectivamente.

Tabela 9. Lacunas de pesquisa identificadas (RQ5).

| Categoria de Lacuna                                                             | Freq. | Artigos representativos                                    |
|---------------------------------------------------------------------------------|-------|------------------------------------------------------------|
| Generalização (domínio, linguagem, escala)                                      | 38    | A01, A17, A28, A30, A35, A36, A40, A41, A49, A53, A54      |
| Arquitetura RAG (granularidade, contexto <i>cross-file</i> , <i>multi-hop</i> ) | 35    | A07, A22, A35, A36, A40, A41, A47, A49, A50, A54, A55, A57 |
| Métricas de avaliação (rigor, reprodutibilidade, <i>benchmarks</i> )            | 26    | A02, A17, A23, A27, A35, A40, A41, A43, A50, A53, A57      |
| Qualidade do <i>knowledge base</i> (atualização, cobertura, ruído)              | 21    | A05, A06, A13, A14, A22, A24, A30, A32, A44, A49, A54, A55 |
| Custo computacional e escalabilidade <i>real-time</i>                           | 14    | A07, A13, A20, A28, A32, A40, A41, A49, A53, A55, A57      |
| Cobertura de linguagens de programação                                          | 13    | A10, A17, A20, A22, A24, A27, A30, A36, A37, A41, A50, A57 |
| Alucinações e falsos positivos                                                  | 10    | A02, A06, A08, A10, A13, A20, A23, A32, A47                |
| Escalabilidade de repositórios                                                  | 9     | A02, A07, A27, A32, A36, A41, A44, A47                     |
| Integração CI/CD e ambientes industriais                                        | 2     | A08, A16                                                   |

A lacuna de generalização manifesta-se em múltiplas dimensões: restrição a uma única linguagem, avaliação em repositórios de pequeno porte sem validação em sistemas industriais de larga escala, e dependência de *datasets* de *benchmark* que podem não refletir a distribuição real de código em produção. A lacuna de arquitetura RAG concentra-se na incapacidade de capturar dependências *cross-file* e inter-procedurais. A lacuna de métricas de avaliação aponta para a ausência de *benchmarks* padronizados específicos para RAG sobre código.

Achado RQ5: Generalização (38 ocorrências) e arquitetura RAG *cross-file* (35) são as lacunas mais críticas, seguidas pela ausência de métricas padronizadas (26). Essas lacunas apontam para as direções de pesquisa mais urgentes do campo.

## 5. DISCUSSÃO

### 5.1 Síntese dos Achados

Os 33 estudos mapeados evidenciam um campo em rápida expansão e progressiva especialização. Quatro tendências transversais emergem da análise conjunta das RQs:

Diversificação arquitetural além do RAG canônico. O RAG denso com *vector store*, embora predominante, coexiste com variantes híbridas, determinísticas, baseadas em grafos e agênticas, cada uma respondendo a limitações específicas do RAG canônico em contextos de código.

Concentração em segurança e vulnerabilidades. Com 9 dos 33 estudos (~27%) focados em detecção de vulnerabilidades, esse é o subdomínio mais desenvolvido, favorecido pela disponibilidade de *datasets* públicos bem estabelecidos como BigVul e PrimeVul.

Dependência de modelos *closed-source*. A dominância de GPT-4 e variantes como geradores principais cria dependência de infraestrutura proprietária e expõe preocupações de privacidade para código proprietário.

Lacuna de avaliação padronizada. A heterogeneidade de métricas, *datasets* e protocolos de avaliação é o obstáculo mais estrutural ao avanço cumulativo do campo.

### 5.2 Implicações para Pesquisa

Com base nas lacunas identificadas, quatro direções de pesquisa prioritárias emergem:

*Benchmarks* padronizados para RAG sobre código: o campo carece de *benchmarks* que avaliem sistematicamente dimensões como raciocínio *cross-file*, sensibilidade à atualização do *knowledge base* e custo computacional.

Raciocínio *cross-file* e inter-procedural: a maioria dos estudos opera no nível de função ou arquivo individual. Abordagens que integrem análise de dependências, grafos de chamada e análise de fluxo de dados em larga escala com RAG representam uma fronteira técnica relevante.

RAG com modelos locais para código proprietário: a predominância de APIs *closed-source* levanta barreiras de privacidade para adoção industrial. Estudos comparativos entre modelos locais e remotos em cenários reais de código proprietário são escassos.

Cobertura multilinguagem: linguagens como Rust, Go, Kotlin e Swift têm presença marginal no corpus, apesar de seu crescimento em sistemas de produção.

### 5.3 Implicações para Praticantes

Para equipes de desenvolvimento que avaliam a adoção de RAG sobre *codebases*, os achados sugerem: (1) *chunking AST-aware* é consistentemente superior ao *chunking* por janela deslizante para código, pois preserva a integridade de unidades sintáticas; (2) *embeddings* híbridos (*dense* + *sparse*) superam *embeddings* puramente densos em tarefas de *code search* com terminologia específica de domínio; (3) para código C/C++ com foco em segurança, abordagens com *knowledge base* estruturada de CWEs e CVEs oferecem vantagem sistemática sobre RAG genérico.

### 5.4 Casos Limítrofes Relevantes

Durante a etapa de elegibilidade, casos ambíguos de CE2 foram decididos com base no seguinte critério: se o código-fonte pudesse ser substituído por qualquer outro artefato estruturado sem que a contribuição central do artigo fosse invalidada, o código não era o objeto do estudo, apenas um corpus indexável, e o artigo foi excluído. Três padrões de exclusão por CE2 merecem registro:

RAG como baseline comparativo negativo (padrão identificado em A51 e outros): estudos onde RAG é explicitamente criticado e usado apenas como condição de controle experimental, com o foco na abordagem alternativa proposta.

RAG como módulo de geração de relatórios em sistema de domínio não-SE (padrão em A52): o módulo RAG-LLM gera relatórios geológicos a partir de dados de caracterização de rocha; o código-fonte da plataforma é a ferramenta, não o objeto.

KG-RAG sobre domínio geral (padrão em A56): framework de calibração para RAG sobre grafos de conhecimento de filmes e entidades Freebase, completamente alheio à Engenharia de Software.

## 6. AMEAÇAS À VALIDADE

As ameaças à validade deste estudo são analisadas em cinco dimensões.

Ameaça 1 - Elaboração da string de busca. A string canônica adotada prioriza precisão sobre sensibilidade: estudos que empregam modelos específicos sem usar os termos "LLM" ou "Large Language Model" nos campos indexados podem não ter sido recuperados. Essa limitação é parcialmente mitigada pela inclusão de "Generative AI" como hiperônimo e pela cobertura complementar das três bases selecionadas.

Ameaça 2 - Ausência de estudos relevantes. Estudos publicados em venues não indexados pelas três bases, ou que empreguem terminologia distinta da adotada na string, podem não ter sido recuperados. *Snowballing* não foi realizado nesta versão do mapeamento, o que constitui uma limitação reconhecida.

Ameaça 3 - Confiabilidade na seleção. A condução por um único pesquisador pode introduzir viés nas etapas de triagem e elegibilidade. Para mitigar esse risco, foram derivados critérios explícitos de inclusão e exclusão estruturados pelo framework GQM, detalhados o suficiente para orientar decisões de triagem de forma objetiva e rastreável. Um critério explícito de decisão foi documentado para casos CE2 ambíguos, conforme descrito na Seção 5.4. A definição rigorosa dos critérios busca mitigar parcialmente esse risco, ainda que não o elimine integralmente.

Ameaça 4 - Confiabilidade na extração de dados. A condução por um único pesquisador caracteriza também uma ameaça à etapa de extração de dados. O registro sistemático de justificativas por artigo ao longo do processo busca minimizar esse risco, ainda que não o elimine integralmente.

Ameaça 5 - Inacessibilidade ao texto completo. Quatro artigos foram excluídos por inacessibilidade (CE8) após esgotamento das alternativas disponíveis (CAPES/UFG, Google Scholar, ResearchGate). É possível que artigos relevantes tenham sido descartados por restrição de acesso, introduzindo viés de seleção na amostra final. Essa prática está alinhada com as diretrizes de Kitchenham e Charters (2007).

## 7. CONCLUSÃO

Este artigo apresentou um Mapeamento Sistemático de Literatura sobre RAG baseado em LLMs aplicado a *codebases* de software, cobrindo o período de 2020 a 2026. A partir de 173 registros recuperados em três bases de dados, 33 estudos primários foram selecionados após processo rigoroso de triagem e leitura integral de 58 artigos, respondendo a cinco questões de pesquisa.

Os principais achados são: (1) detecção de vulnerabilidades é a tarefa-alvo dominante (~27% dos estudos), seguida por Q&A sobre *codebase* e *code completion*; (2) a família GPT da OpenAI domina como componente gerador, com emergência crescente de modelos *open-source* competitivos; (3) Python e Java são as linguagens mais estudadas, com C/C++ concentrado em segurança de sistemas; (4) métricas de classificação (P/R/F1) dominam a avaliação, com ausência de *benchmarks* padronizados para o domínio; (5) generalização, arquitetura RAG *cross-file* e avaliação padronizada são as categorias de lacunas mais críticas.

As tendências arquiteturais mais relevantes são a emergência do RAG determinístico sem *embeddings*, o crescimento do RAG *agentic multi-step* e a adoção de *chunking AST-aware*. Essas tendências indicam um campo que está desenvolvendo abordagens especializadas que exploram a estrutura formal das linguagens de programação, superando a fase de adaptação direta de técnicas de RAG para texto.

Como trabalhos futuros, destacam-se: a realização de *snowballing* para ampliar a cobertura do mapeamento; a inclusão de bases complementares como Springer Link e Web of Science; a extensão do período de cobertura; e a condução de uma revisão sistemática com meta-análise sobre subdomínios específicos, como detecção de vulnerabilidades com RAG, onde o volume de estudos já justifica síntese quantitativa mais aprofundada.

## Referências

- BASIL, V. R. Software Modeling and Measurement: The Goal/Question/Metric Paradigm. Institute for Advanced Computer Studies, University of Maryland, 1992.
- BASIL, V. R.; WEISS, D. M. A Methodology for Collecting Valid Software Engineering Data. IEEE Transactions on Software Engineering, v. SE-10, n. 6, p. 728-738, 1984.
- DYBA, T.; KITCHENHAM, B.; JORGENSEN, M. Evidence-based software engineering for practitioners. IEEE Software, v. 22, n. 1, p. 58-65, 2005.

- FENG, Z. et al. CodeBERT: A Pre-Trained Model for Programming and Natural Languages. In: Findings of EMNLP, 2020.
- GUO, D. et al. UniXcoder: Unified Cross-Modal Pre-Training for Code Representation. In: ACL, 2022.
- KITCHENHAM, B.; CHARTERS, S. Guidelines for Performing Systematic Literature Reviews in Software Engineering. EBSE Technical Report EBSE-2007-01, Keele University, v. 2.3, 2007.
- LEWIS, P. et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In: NeurIPS, 2020.
- LI, R. et al. StarCoder: May the Source Be with You! Transactions on Machine Learning Research, 2023.
- OUZZANI, M. et al. Rayyan: a web and mobile app for systematic reviews. Systematic Reviews, 2016.
- PETERSEN, K.; VAKKALANKA, S.; KUZNIARZ, L. Guidelines for conducting systematic mapping studies in software engineering: An update. Information and Software Technology, v. 64, p. 1-18, 2015.
- ROZIÈRE, B. et al. Code Llama: Open Foundation Models for Code. arXiv:2308.12950, 2023.
- VASWANI, A. et al. Attention Is All You Need. In: NeurIPS, 2017.

### Referências dos Estudos Primários

- [A01] YANG et al. A Deep Dive into Retrieval-Augmented Generation for Code Completion: Experience on WeChat. In: ACL, 2025.
- [A02] IONESCU, A. C.; TITOV, S.; IZADI, M. A Multi-agent Onboarding Assistant based on Large Language Models, Retrieval Augmented Generation, and Chain-of-Thought. In: FSE Companion '25: 33rd ACM International Conference on the Foundations of Software Engineering, p. 1208-1212. ACM, 2025.
- [A05] DARNELL et al. An Empirical Comparison of Code Generation Approaches for Ansible. In: 2nd IEEE/ACM International Workshop on Interpretability, Robustness, and Benchmarking in Neural Software Engineering (InteNSE), 2024. IEEE, 2024.
- [A06] SAJU, A.; MUHTADI, C. S.; AZIM, A. An Empirical Evaluation of LLM-Based Approaches for Code Vulnerability Detection: RAG, SFT, and Dual-Agent Systems. In: 35th International Conference on Collaborative Advances in Software and Computing (CASCON), 2025. IEEE, 2025.
- [A07] ASWIN DEV et al. Analysing Code-Based Retrieval Augmented Generation Methods for Knowledge Retention. IEEE Access, 2025.
- [A08] ABTAHI, A.; AZIM, A. Augmenting Large Language Models with Static Code Analysis for Automated Code Quality Improvements. In: 2nd ACM International Conference on AI Foundation Models and Software Engineering (FORGE 2025), co-located with ICSE 2025. IEEE/ACM, 2025.
- [A10] NAULTY et al. Bugdar: AI-Augmented Secure Code Review for GitHub Pull Requests. In: 2025 IEEE Conference on Artificial Intelligence (CAI 2025), Santa Clara, CA, USA. IEEE, 2025.
- [A13] TU, X.; ZHANG, Y.; CHEN, B. Collaborative Agent Framework for Web Vulnerability Discovery in Source Code. In: IEEE ICCWAMTIP, 2025.
- [A14] IBIYO et al. Detecting Malicious Source Code in PyPI Packages with LLMs: Does RAG Come in Handy. In: EASE, 2025.
- [A16] RAJAGURU et al. Enhancing Flutter App Development: Addressing Configuration and Compatibility Bugs Using Large Language Models. In: IEEE SCSE, 2025.
- [A17] HE et al. Enhancing the ability of LLMs for spaceborne equipment code generation via retrieval-augmented generation and contrastive learning. Automated Software Engineering, Springer, 2026.
- [A20] ANTAL et al. Evaluating Retrieval-Augmented Generation for LLM-Based Vulnerability Detection: An Empirical Study on Real-World Java Vulnerabilities. IEEE Access, 2026.
- [A22] LAYEGH, A.; PAYBERAH, A. H.; MATSKIN, M. From Code to Execution: Multi-Agent GraphRAG for Automated Artifact Generation. In: EuroMLSys, ACM, 2026.
- [A23] OLEWICKI et al. Impact of an LLM-based Review Assistant in Practice: A Mixed Open-/Closed-source Case Study. IEEE Transactions on Software Engineering, 2026.
- [A24] WANG et al. Integrating Retrieval-Augmented Generation for Enhanced Code Reuse: A Comprehensive Framework for Efficient Software Development. In: IEEE SWC, 2024.
- [A27] OKUTAN et al. Leveraging RAG-LLM to Translate C++ to Rust. In: ICAA, IEEE, 2024.

- [A28] OUCHEBARA, D. S.; DUPONT, S. Llama-Based Source Code Vulnerability Detection: Prompt Engineering vs Fine Tuning. In: NICOMETTE, V. et al. (eds.) Computer Security - ESORICS 2025. Lecture Notes in Computer Science, v. 16053, p. 289-308. Springer, Cham, 2026.
- [A30] QAYYUM et al. LLM-assisted Bug Identification and Correction for Verilog HDL. ACM Transactions on Design Automation of Electronic Systems, v. 30, 2025.
- [A32] HU et al. Local Agentic RAG-Based Information System Development for Intelligent Analysis of GitHub Code Repositories in Computer Science Education. International Journal of Modern Education and Computer Science, v. 17, 2025.
- [A35] STRICH et al. On Improving Repository-Level Code QA for Large Language Models. In: ACL Student Research Workshop, 2024.
- [A36] BEN HAJHMIDA, M.; LEE, E. A. RAG and Agentic Assistant: A Combined Approach. In: AISoLA, LNCS 16220, 2026.
- [A37] BOCHENEK, J.; PROTASIEWICZ, J.; PEDRYCZ, W. RAGdeterm: Deterministic retrieval-augmented generation for code generation. SoftwareX, v. 34, 2026.
- [A40] RAFIEI OSKOOEI et al. Repository-Level Code Understanding by LLMs via Hierarchical Summarization: Improving Code Search and Bug Localization. In: LNCS 15886, 2025.
- [A41] SHEN et al. Repository-level code completion with adaptive segmentation and fused retrieval. Empirical Software Engineering, v. 31, n. 26, 2026.
- [A43] SHAWON et al. Retrieval Augmented Generation Fine-Tuned LLM Model for Code Recommendations to Mitigate Lock Contention. In: ICPE Companion, ACM, 2025.
- [A44] TSAI, Y.; LIU, Y.; REN, H. RTLFixer: Automatically Fixing RTL Syntax Errors with Large Language Models. In: DAC, ACM/IEEE, 2024.
- [A47] SHAIKH et al. Secure Retrieval-Augmented Generation Framework for Automated Knowledge Access in Enterprise Git Repositories. In: IC3ET, IEEE, 2026.
- [A49] WEI et al. SmartAuditFlow: A Dynamic Plan-Execute Framework for Advanced Smart Contract Security Analysis. ACM Transactions on Software Engineering and Methodology, 2025.
- [A50] AMIN et al. Source Code Error Categorization and Explanation by Retrieval Augmented Generation. In: AAIML, IEEE, 2026.
- [A53] WEI et al. Unleashing the Power of LLM to Infer State Machine from the Protocol Implementation. In: IWQoS, IEEE, 2025.
- [A54] CORREIA et al. Unveiling the Potential of a Conversational Agent in Developer Support: Insights from Mozilla's PDF.js Project. In: AIware, ACM, 2024.
- [A55] LIAO et al. VulKnow: Enhancing Vulnerability Detection with Structured Knowledge and Large Language Models. IEEE Internet of Things Journal, 2026.
- [A57] WANG et al. Wired for Reuse: Automating Context-Aware Code Adaptation in IDEs via LLM-Based Agent. In: ASE, IEEE/ACM, 2025.